```
tailwindcss -i ./app/assets/stylesheets/application.tailwind.css
-o ./app/assets/builds/application.css"
```

This calls the Tailwind command-line tool, tells it to look at the application.tailwind.css, build itself, and then output to /app/assets/builds/application.css.

To use this file, we link it in our app/views/layouts/application.html.erb with the helper method stylesheet_link_tag:

```
<%= stylesheet_link_tag "application", "data-turbo-track": "reload" %>
```

We're all set.

At this point, we can get new CSS into the system in a couple of ways. We can add a previously unused Tailwind utility class to any of the files covered by the content entry in the configuration file. Changing one of those files triggers the watch mode, causing the Tailwind CLI to recompile files and produce a new CSS file for the browser that includes the new utility.

If we want to write custom CSS and we still don't want to upgrade to PostCSS or Sass, we have three simple options:

- We can add new CSS to the application.tailwind.css, where it will get built by the Tailwind CLI and included in the application.css bundle.
- We can create a new CSS file in application/builds/css and link it by adding it as an argument to the stylesheet_link_tag helper. This works as long as the new file doesn't need Tailwind.
- We can create a new CSS file and import it in our JavaScript and build it via esbuild. This is how we'll integrate with tools that package both JavaScript and CSS, and also as we'll see it requires some small configuration changes.

There are some more complex options that we'll look at in Chapter 7, *Bundling*.